

附录 A：架构地图

1. 当前技术栈

- Expo 54
- React Native 0.81
- React 19
- TypeScript strict
- AsyncStorage
- 无后端、无登录、无数据库、无云同步

移动端命令入口在 `D:\imcfo\mobile\package.json` :

```
cd D:\imcfo\mobile
npm.cmd install
npm.cmd run typescript
npm.cmd run web --port 8091 --host localhost
npm.cmd start
npm.cmd run android
```

2. 关键目录

```
index.ts
App.ts
App
App
useAppbar.ts
components
AppIcon.ts
FinanceUI.ts
InAllDownIncomeSnakesSection.ts
charts
domain
```

models

accounting

reports

transactionRules

hooks

screens

storage

types

utils

3. 数据流

screens

useAppData

transactionRules / reconciliationRules

asyncStorageAdapters

AsyncStorage

screens

domain/reports or domain/accounting/calculations

view model

UI components

4. 模块职责

- `App.tsx` : 页面路由、底部导航、跨页面 props 组装。
- `useAppData.ts` : 应用数据门面，屏幕只通过它读写业务数据。
- `storage/*` : 本地存储边界，隐藏 `AsyncStorage` 细节。
- `domain/models/*` : `typed data model`。
- `domain/accounting/calculations.ts` : 基础报表公式和 Dashboard summary。
- `domain/accounting/transactionRules.ts` : 交易输入到财务状态变更。
- `domain/accounting/reconciliationRules.ts` : 账户和资产对账调整。
- `domain/accounting/periodFilters.ts` : 报表期间过滤。
- `domain/reports/*` : 三大报表和分析报告 `view model`。
- `screens/*` : 页面展示、交互状态、弹层和导航。

- `components/financeUI.tsx` : 当前未提交的共享金融 UI primitives。
- `styles/theme.ts` : 视觉 token。

5. 已知架构风险

- `DashboardScreen.tsx` 内仍有部分趋势、现金流方向、资产/负债构成等聚合逻辑，后续应迁入 `domain/report engine`。
- `operatingAnalysisReport.ts` 和 `profitabilityAnalysis.ts` 是静态 mock 数据，不应被当成真实计算引擎。
- `calculateCashFlowByType` 与交易展示索引的现金流入判断存在口径漂移风险。
- 应收/应付现金流分类在文档和实现之间需要再统一。
- 当前没有自动化公式测试，只有 TypeScript typecheck。

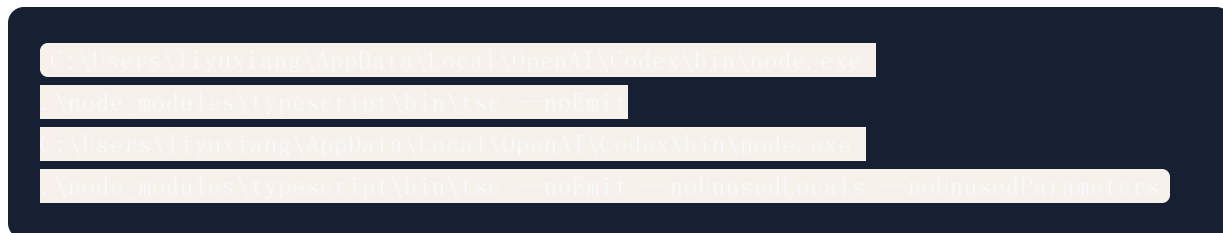
6. 2026-05-05 审计同步

最新上下文快照记录了一轮 overnight-quality 维护审计，重点检查移动端编译稳定性、交易记录状态安全、未使用代码和高复杂度演示数据不变量。

记录到交接包的最新实现方向：

- 交易记录页继续统一使用 `transactionDisplayIndex` 作为展示索引来源。
- 交易记录详情选中项会在 `records index` 变化后清理，降低导入、清空、重置数据后的 stale record 风险。
- 交易记录默认保留月份懒加载策略，搜索或筛选时才 hydration 全量 records。
- 若继续维护交易记录页，应避免重新引入旧版本地搜索/分组 helper。
- 若继续维护分析报告页，应优先把静态 mock 数据接入真实 AppData 和三大报表口径。

验证记录：



结果：均通过。